In [1]:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

import nltk
from nltk.corpus import stopwords
from nltk.util import ngrams
from nltk.tokenize import word_tokenize,sent_tokenize
from nltk import pos_tag
from nltk.corpus import wordnet

import re

import string
from string import punctuation

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn import metrics
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import classification_report,confusion_matrix,accuracy
from sklearn.preprocessing import LabelBinarizer

from bs4 import BeautifulSoup

import unicodedata
```

In [2]:

```
dataset = pd.read_json('/Users/moe/Desktop/HuffPost_News_Dataset_Categories
dataset.drop(['authors', 'link', 'date'], axis = 1, inplace = True)
dataset.head()
```

Out[2]:

	category	headline	short_description
0	CRIME	There Were 2 Mass Shootings In Texas Last Week...	She left her husband. He killed their children...
1	ENTERTAINMENT	Will Smith Joins Diplo And Nicky Jam For The 2...	Of course it has a song.
2	ENTERTAINMENT	Hugh Grant Marries For The First Time At Age 57	The actor and his longtime girlfriend Anna Ebe...
3	ENTERTAINMENT	Jim Carrey Blasts 'Castrato' Adam Schiff And D...	The actor gives Dems an ass-kicking for not fi...
4	ENTERTAINMENT	Julianna Margulies Uses Donald Trump Poop Bags...	The "Dietland" actress said using the bags is ...

In [3]:

```
categories = dataset['category'].value_counts().index

def groupper(grouplist,name):
    for ele in categories:
        if ele in grouplist:
            dataset.loc[dataset['category'] == ele, 'category'] = name
```

In [4]:
```
groupper( grouplist= ['WELLNESS', 'HEALTHY LIVING'] , name =  'HEALTH AND W

groupper( grouplist= ['SPORTS','ENTERTAINMENT' , 'COMEDY','WEIRD NEWS','ART

groupper( grouplist= ['BUSINESS' ,  'MONEY','TECH', 'SCIENCE'] , name =  'B

groupper( grouplist= ['PARENTING', 'PARENTS' ,'EDUCATION' ,'COLLEGE','TRAVE
```

In [5]:
```
shuffled_df = dataset.sample(frac=1,random_state=4)

politics = shuffled_df.loc[shuffled_df['category'] == 'POLITICS']
sports = shuffled_df.loc[shuffled_df['category'] == 'SPORTS AND ENTERTAINME
health = shuffled_df.loc[shuffled_df['category'] == 'HEALTH AND WELLNESS']
businessandscience = shuffled_df.loc[shuffled_df['category'] == 'BUSINESS A
general = shuffled_df.loc[shuffled_df['category'] == 'GENERAL']

politics_normal = shuffled_df.loc[shuffled_df['category'] == 'POLITICS'].sa
sports_normal = shuffled_df.loc[shuffled_df['category'] == 'SPORTS AND ENTE
health_normal = shuffled_df.loc[shuffled_df['category'] == 'HEALTH AND WELI

normalized_df = pd.concat([politics_normal , sports_normal , health_normal
```

In [6]:
```
normalized_df.duplicated().sum()
```

Out[6]: 59

In [7]:
```
normalized_df.drop_duplicates(keep='last', inplace=True)
```

In [8]:
```
normalized_df.loc[normalized_df['headline'] == "", 'headline'] = np.nan
normalized_df.dropna(subset=['headline'], inplace=True)
print(len(normalized_df[normalized_df['headline'] == ""]))
```

0

In [9]:
```
normalized_df.loc[normalized_df['short_description'] == "", 'short_descript
normalized_df.dropna(subset=['short_description'], inplace=True)
print(len(normalized_df[normalized_df['short_description'] == ""]))
```

0

In [10]:
```
normalized_df['text'] = normalized_df['headline']+" "+normalized_df['short_
normalized_df.drop(columns =['headline','short_description'],axis = 1, inpl
normalized_df.head()
```

Out[10]:

	category	text
87800	POLITICS	Letter to My Sons on Father's Day That's what ...
20916	POLITICS	Exxon Fires Back Over Fine For Violating Russi...
5562	POLITICS	Donald Trump Puts Focus On Mental Health In Fi...
33767	POLITICS	National Security Council Spokesman Resigns Ov...
1498	POLITICS	Michigan Gov. Candidate Delayed Rescue Of Anim...

In [11]:

```
from sklearn.utils import shuffle
normalized_df = shuffle(normalized_df)
normalized_df.reset_index(inplace=True, drop=True)
```

In [12]:

```
stop = set(stopwords.words('english'))
punctuation = list(string.punctuation)
stop.update(punctuation)

def strip_html(text):
    soup = BeautifulSoup(text, "html.parser")
    return soup.get_text()
def remove_between_square_brackets(text):
    return re.sub(r'\[[^]]*\]', '', text)
def remove_urls(text):
    return re.sub(r'http\S+', '', text)
def remove_stopwords(text):
    final_text = []
    for i in text.split():
        if i.strip().lower() not in stop:
            final_text.append(i.strip())
    return " ".join(final_text)
def clean_text(text):
    text = re.sub(r'\[.*?\]', '', text)
    text = re.sub(r'https?://\S+|www\.\S+', '', text)
    text = re.sub(r'<.*?>+', '', text)
    text = re.sub(r'[%s]' % re.escape(string.punctuation), '', text)
    text = re.sub(r'\n', '', text)
    text = re.sub(r'\w*\d\w*', '', text)
    text = re.sub(r'caption','',text)
    text = re.sub(r'photo','',text)
    text = re.sub(r'containerid','',text)
    text = re.sub(r'function','',text)
    text = re.sub(r'modernizr','',text)
    text = re.sub(r'modernizrphone','',text)
    text = re.sub(r'modernizrmobile','',text)
    text = re.sub(r'modernizrtablet','',text)
    text = re.sub(r'typeof','',text)
    text = re.sub(r'videopinner','',text)
    text = re.sub(r'undefined','',text)
    text = re.sub(r'null','',text)
    text = str(text).lower()
    return text

def denoise_text(text):
    text = strip_html(text)
    text = remove_between_square_brackets(text)
    text = remove_urls(text)
    text = remove_stopwords(text)
    text = clean_text(text)
    return text
```

In [13]:

```
normalized_df['text']=normalized_df['text'].apply(denoise_text)
```

In [14]:

```
dataset = normalized_df
```

In [15]:

```
print("We have a total of {} categories now".format(dataset['category'].nur
dataset['category'].value_counts()
```

```
We have a total of 4 categories now
```

Out[15]:
```
HEALTH AND WELLNESS          14140
POLITICS                     13613
SPORTS AND ENTERTAINMENT     12464
BUSINESS AND SCIENCE         10588
Name: category, dtype: int64
```

In [16]:
```
df = dataset.copy()
```

In [17]:
```
def lemmatize_words(text):
    wnl = nltk.stem.WordNetLemmatizer()
    lem = ' '.join([wnl.lemmatize(word) for word in text.split()])
    return lem

df['text'] = df['text'].apply(lemmatize_words)
```

In [20]:
```
categories = df.groupby('category').size().index.tolist()
category_int = {}
int_category = {}
for i, k in enumerate(categories):
    category_int.update({k:i})
    int_category.update({i:k})

df['c2id'] = df['category'].apply(lambda x: category_int[x])
```

In [31]:
```
category_int
```

Out[31]:
```
{'BUSINESS AND SCIENCE': 0,
 'HEALTH AND WELLNESS': 1,
 'POLITICS': 2,
 'SPORTS AND ENTERTAINMENT': 3}
```

In [22]:
```
x_train,x_test,y_train,y_test = train_test_split(df.text,df.c2id,random_sta
```

In [26]:
```
count_vectorizer = CountVectorizer(stop_words='english')
x_train = count_vectorizer.fit_transform(x_train.values)
x_test = count_vectorizer.transform(x_test.values)
```

MODEL

Using 2 different models with 2 different parameters for parameter investigation values of alpha and c

Naive Bayes

In [35]:

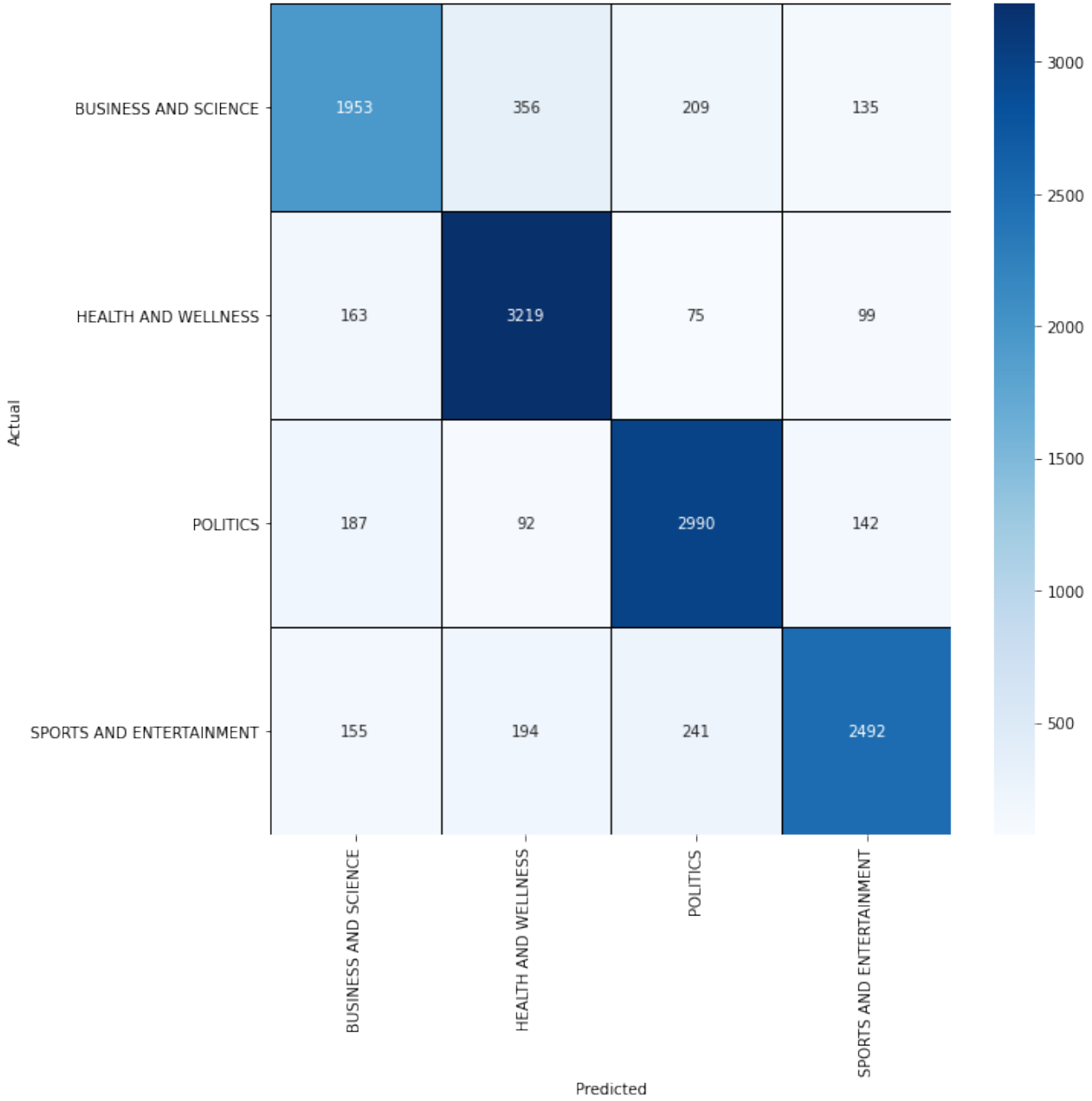
```
#Model 1 - default parameters
from sklearn.metrics import classification_report

nb_classifier1 = MultinomialNB()
nb_classifier1.fit(x_train, y_train)

pred1 = nb_classifier1.predict(x_test)

print(classification_report(y_test, pred1, target_names = ['BUSINESS AND SC
```

```
                         precision    recall  f1-score   support

    BUSINESS AND SCIENCE       0.79      0.74      0.76      2653
     HEALTH AND WELLNESS       0.83      0.91      0.87      3556
                POLITICS       0.85      0.88      0.86      3411
SPORTS AND ENTERTAINMENT       0.87      0.81      0.84      3082

                accuracy                           0.84     12702
               macro avg       0.84      0.83      0.83     12702
            weighted avg       0.84      0.84      0.84     12702
```

In [36]:

```
cm1 = confusion_matrix(y_test,pred1)
cm1 = pd.DataFrame(cm1 , index = ['BUSINESS AND SCIENCE','HEALTH AND WELLNE
plt.figure(figsize = (10,10))
sns.heatmap(cm1,cmap= "Blues", linecolor = 'black' , linewidth = 1 , annot
plt.yticks(rotation=0)
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

Out[36]: Text(68.99999999999999, 0.5, 'Actual')



In [38]:

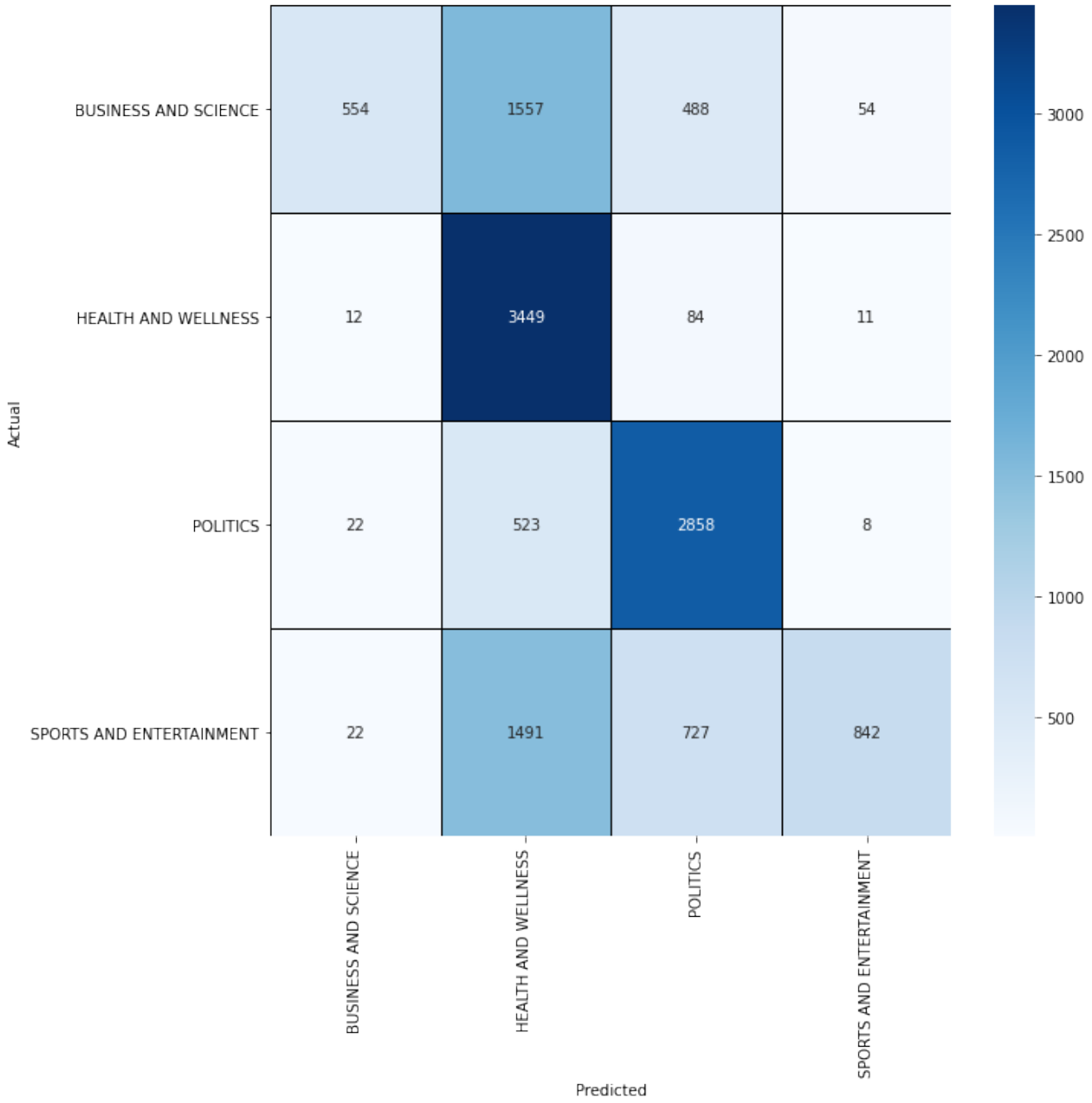
```
#Model 2
nb_classifier2 = MultinomialNB(alpha = 1000)
nb_classifier2.fit(x_train, y_train)

pred2 = nb_classifier2.predict(x_test)

print(classification_report(y_test, pred2, target_names = ['BUSINESS AND SC
```

	precision	recall	f1-score	support
BUSINESS AND SCIENCE	0.91	0.21	0.34	2653
HEALTH AND WELLNESS	0.49	0.97	0.65	3556
POLITICS	0.69	0.84	0.76	3411
SPORTS AND ENTERTAINMENT	0.92	0.27	0.42	3082
accuracy			0.61	12702
macro avg	0.75	0.57	0.54	12702
weighted avg	0.74	0.61	0.56	12702

In [41]:

```
cm2 = confusion_matrix(y_test,pred2)
cm2 = pd.DataFrame(cm2 , index = ['BUSINESS AND SCIENCE','HEALTH AND WELLNE
plt.figure(figsize = (10,10))
sns.heatmap(cm2,cmap= "Blues", linecolor = 'black' , linewidth = 1 , annot
plt.yticks(rotation=0)
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

Out[41]: Text(68.99999999999999, 0.5, 'Actual')



Support Vector Machine (SVM)

In [42]:

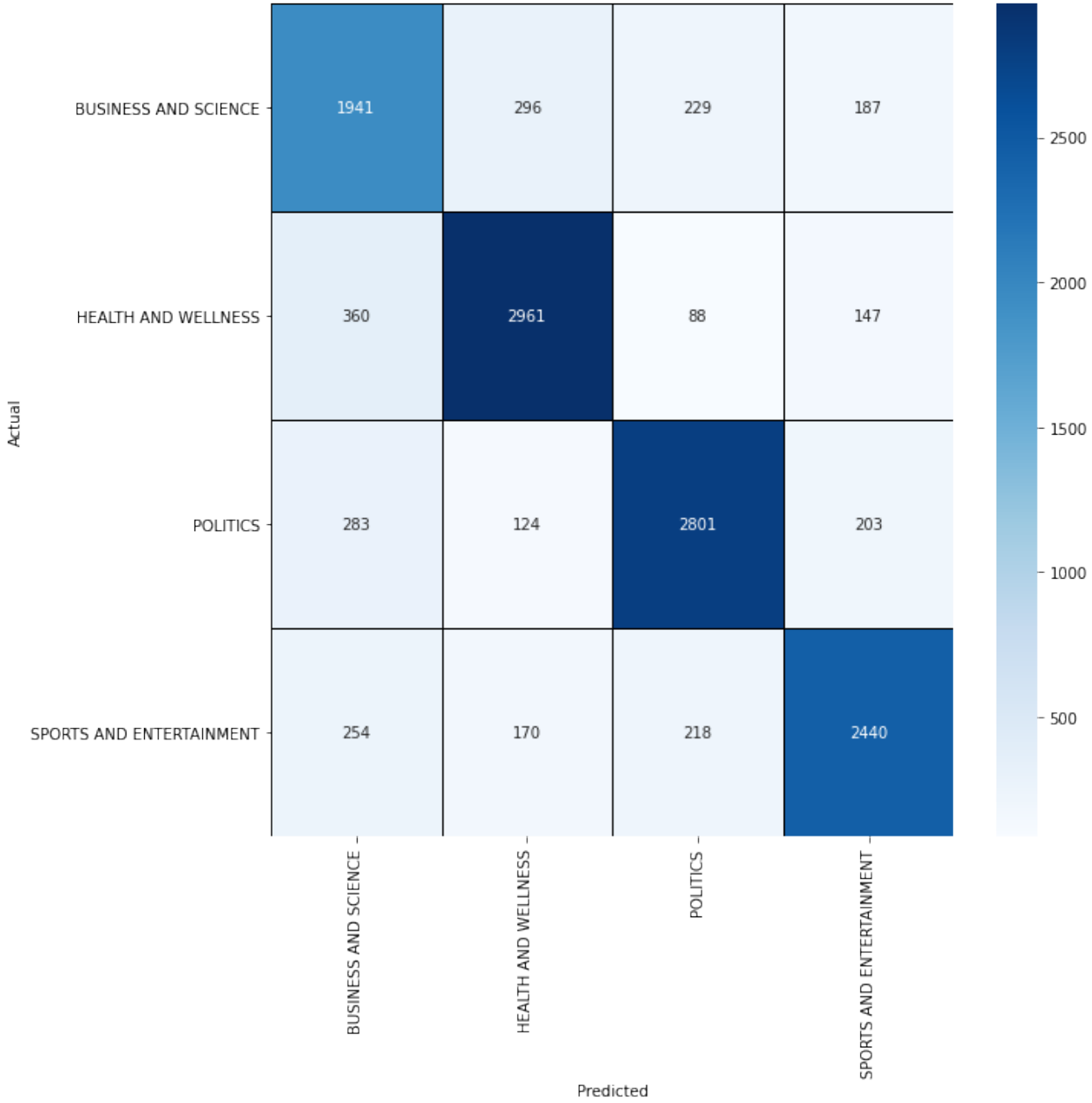
```
#Model 1
from sklearn.svm import SVC

svc_model1 = SVC(C=1, kernel='linear', gamma= 1)
svc_model1.fit(x_train, y_train)

prediction1 = svc_model1.predict(x_test)

print(classification_report(y_test, prediction1, target_names = ['BUSINESS
```

```
                         precision    recall  f1-score   support

    BUSINESS AND SCIENCE       0.68      0.73      0.71      2653
     HEALTH AND WELLNESS       0.83      0.83      0.83      3556
                POLITICS       0.84      0.82      0.83      3411
SPORTS AND ENTERTAINMENT       0.82      0.79      0.81      3082

                accuracy                           0.80     12702
               macro avg       0.79      0.79      0.79     12702
            weighted avg       0.80      0.80      0.80     12702
```

In [43]:

```
cm3 = confusion_matrix(y_test,prediction1)
cm3 = pd.DataFrame(cm3 , index = ['BUSINESS AND SCIENCE','HEALTH AND WELLNE
plt.figure(figsize = (10,10))
sns.heatmap(cm3,cmap= "Blues", linecolor = 'black' , linewidth = 1 , annot
plt.yticks(rotation=0)
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

Out[43]: Text(68.99999999999999, 0.5, 'Actual')



In [44]:

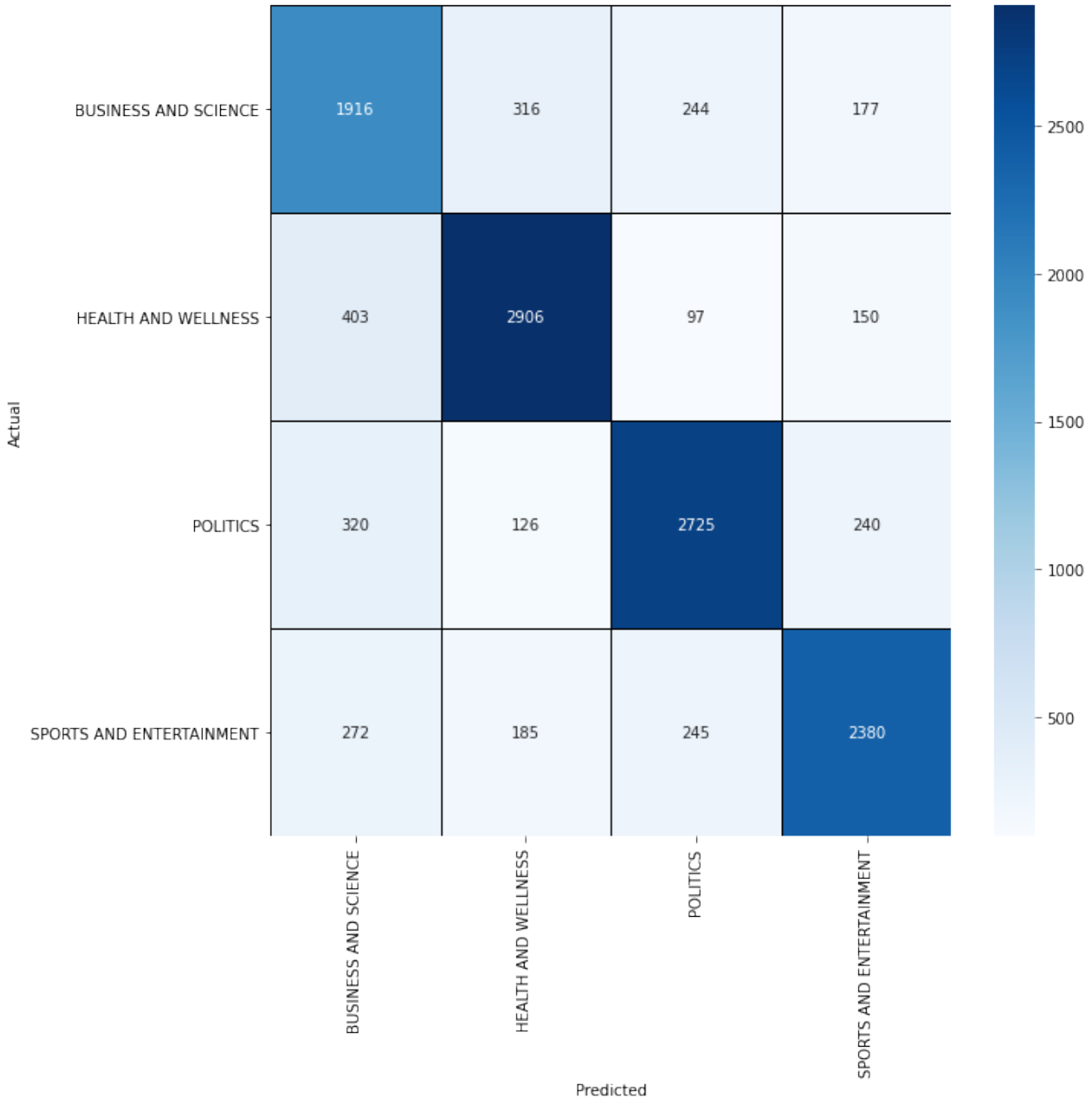
```
#Model 2
svc_model2 = SVC(C= 100, kernel='linear', gamma= 1)
svc_model2.fit(x_train, y_train)

prediction2 = svc_model2.predict(x_test)

print(classification_report(y_test, prediction2, target_names = ['BUSINESS
```

	precision	recall	f1-score	support
BUSINESS AND SCIENCE	0.66	0.72	0.69	2653
HEALTH AND WELLNESS	0.82	0.82	0.82	3556
POLITICS	0.82	0.80	0.81	3411
SPORTS AND ENTERTAINMENT	0.81	0.77	0.79	3082
accuracy			0.78	12702
macro avg	0.78	0.78	0.78	12702
weighted avg	0.78	0.78	0.78	12702

In [45]:

```
cm4 = confusion_matrix(y_test,prediction2)
cm4 = pd.DataFrame(cm4 , index = ['BUSINESS AND SCIENCE','HEALTH AND WELLNE
plt.figure(figsize = (10,10))
sns.heatmap(cm4,cmap= "Blues", linecolor = 'black' , linewidth = 1 , annot
plt.yticks(rotation=0)
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

Out[45]: Text(68.99999999999999, 0.5, 'Actual')



In [ ]: